

Java OOP Principles - Cheat Sheet

OOP is how Java organizes code around objects instead of loose functions. Four principles hold it together, and the easy way to remember them is **A PIE**.

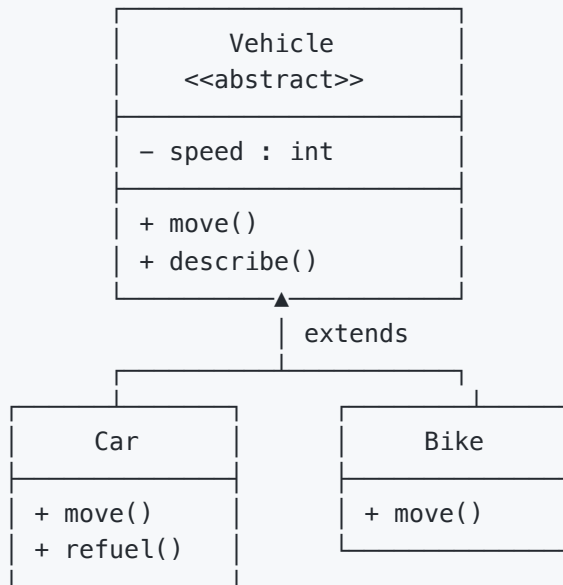
A PIE: the four principles

Letter	Principle	In one line
A	Abstraction	Hide the implementation, expose only what matters
P	Polymorphism	One method name, different behaviour per type
I	Inheritance	Reuse a parent's state and behaviour through an IS-A relationship
E	Encapsulation	Hide the data, expose behaviour through methods

This sheet uses one running example the whole way through: a `Vehicle` and its subtypes. Each section builds on the last, so you are not restarting the mental model four times.

Code is shortened for clarity. Imports and unrelated members are left out, and each snippet is one view of the same `Vehicle`, `Car`, `Bike`, and `Engine` classes.

The full model, as a UML class diagram:



The arrow with the hollow head points from child to parent. That is the UML way to read “Car IS-A Vehicle”.

1. Abstraction

Show what an object can do, hide how it does it.

Why it exists, and this is what separates it from encapsulation:

- **Hide complexity.** The caller drives a `Vehicle` without knowing how the engine works.
- **Expose only the necessary operations.** A small surface is easier to use and harder to misuse.
- **Allow multiple implementations.** `Car`, `Bike`, and `Truck` all satisfy the same contract in their own way.

In Java, abstraction shows up as abstract classes and interfaces.

Abstract class for partial implementation that can hold state:

```
public abstract class Vehicle {
    private int speed;                // encapsulated state, used later

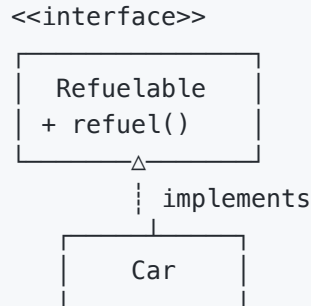
    public abstract void move();      // each subtype decides how

    public void describe() {         // shared, concrete behaviour
        System.out.println("moving at " + speed);
    }
}
```

Interface for a pure contract, no instance state:

```
public interface Refuelable {
    void refuel();
}
```

Now use the interface in the running example. `Car` is a `Vehicle` and is also `Refuelable` :



```
public class Car extends Vehicle implements Refuelable {
    @Override
    public void move() { System.out.println("driving on the road"); }
    @Override
    public void refuel() { System.out.println("filling the tank"); }
}
```

Program against the contract, not the concrete type:

```
Refuelable fuel = new Car();
fuel.refuel(); // works for any Refuelable
```

Takeaway: this is abstraction and polymorphism together. The variable's type is the contract, the behaviour comes from the real object behind it.

Picking one: interface when you only care what an object can do, abstract class when subtypes share state or partial logic. Since Java 8 interfaces can have default methods, and since Java 9 also static and private methods. They still hold no instance fields.

2. Polymorphism

Same method name, different behaviour based on the actual object. Java has two kinds, and interviews expect you to name them precisely.

Compile-time (overloading): same name, different parameters. The compiler picks the method.

```
void refuel() { /* fill to full */ }
void refuel(int litres) { /* fill a set amount */ }
```

Runtime (overriding): a subtype redefines a parent method. The JVM picks the method at runtime based on the real object. This is dynamic dispatch.

```
Vehicle[] vehicles = { new Car(), new Bike(), new Truck() };

for (Vehicle v : vehicles) {
    v.move();    // same call, different behaviour per type
}
// Car    -> "driving on the road"
// Bike   -> "pedaling along"
// Truck  -> "hauling the load"
```

Takeaway: you write code against `Vehicle` and stop caring which subtype shows up. Always add `@Override`. It catches typos like `movee()` at compile time.

From class to runtime

Dynamic dispatch is easier to hold once you see the object's life, from a class you wrote to a method call resolved at runtime:

```
Class    (the blueprint you wrote: Vehicle, Car)
  |
  ▼
Object      Vehicle v = new Car();
  |
  ▼
Method call  v.move();
  |
  ▼
Runtime polymorphism  Car.move() runs, chosen by the real type
```

3. Inheritance

A class extends another and reuses its fields and methods. Use it only when the relationship is genuinely IS-A. A `Car` is a `Vehicle`, so `Car` extends `Vehicle`.

```
public class Car extends Vehicle {
    @Override
    public void move() { System.out.println("driving on the road"); }
}

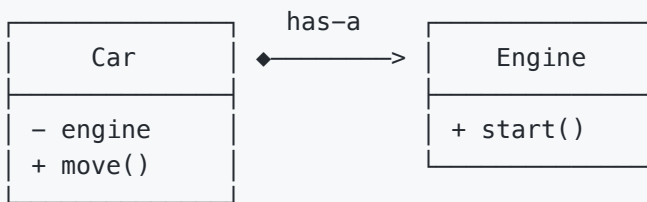
public class Bike extends Vehicle {
    @Override
    public void move() { System.out.println("pedaling along"); }
}
```

Things to keep in mind:

- Java allows single inheritance only. One class, one parent.
- Use `super(...)` to call the parent constructor or method.
- Constructors are not inherited. The child calls the parent through `super(...)`.

Composition over inheritance

If the relationship is HAS-A, not IS-A, hold the other object instead of extending it. A `Car` is not an `Engine`, it has one. In UML, the filled diamond marks composition:



```
public class Car extends Vehicle implements Refuelable {
    private final Engine engine; // Car HAS-A Engine

    public Car(Engine engine) {
        this.engine = engine;
    }

    @Override
    public void move() {
        engine.start();
        System.out.println("driving on the road");
    }
}

class Engine {
    void start() { System.out.println("engine on"); }
}
```

Takeaway: if you find yourself extending a class only to reuse one method, reach for composition instead.

4. Encapsulation

Hide the data, expose the behaviour. Fields go `private`, and outside code touches them only through methods. This keeps internal state safe and gives you one place to add validation.

```

public abstract class Vehicle {
    private int speed;                // no direct outside access

    public int getSpeed() {
        return speed;
    }

    public void accelerate(int delta) {
        if (delta <= 0) {
            throw new IllegalArgumentException("delta must be positive");
        }
        speed += delta;              // the only way speed changes
    }
}

```

Why it matters:

- You can change how `speed` is stored later without breaking callers.
- Validation sits in one place, `accelerate`, not scattered everywhere.

Takeaway: making fields `public` for convenience throws all of this away. Either expose them through methods or stop exposing them at all.

Encapsulation vs Abstraction

The most common interview trip-up, because both are about hiding. They hide different things.

Aspect	Encapsulation	Abstraction
Hides	Data and internal state	Implementation and complexity
Achieved with	private fields, getters, setters	interface, abstract class
Question it answers	How is the state protected?	What can this object do?
Goal	Safety and control	Simplicity, and many implementations

Short version: encapsulation hides the data, abstraction hides the how.

When to use what

Use	When
Encapsulation	Always, whenever you hold state worth protecting
Inheritance	Only for a true IS-A relationship
Composition	Whenever you just need to reuse behaviour (HAS-A)
Interface	For a contract that unrelated types implement
Abstract class	When subtypes share state or partial code

All four in one place

The same `Car` shows every principle at once. This is the picture to keep.

```
Engine engine = new Engine();
Vehicle v = new Car(engine);    // abstraction: typed as Vehicle
v.accelerate(20);               // encapsulation: speed changes only here
v.move();                       // polymorphism: Car.move() runs
```

Principle	Where it shows in this snippet
Encapsulation	<code>speed</code> is <code>private</code> , changed only through <code>accelerate</code>
Inheritance	<code>Car</code> extends <code>Vehicle</code> and reuses <code>accelerate</code> and <code>describe</code>
Polymorphism	<code>v</code> is a <code>Vehicle</code> , but <code>Car.move()</code> runs at runtime
Abstraction	<code>Vehicle</code> defines <code>move()</code> , <code>Car</code> decides how, <code>Engine</code> stays hidden

Quick reference

Concept	Keyword	Mental model
Abstraction	<code>abstract</code> , <code>interface</code>	Steering wheel hides the engine
Polymorphism	<code>@Override</code> , overloading	One name, many behaviours
Inheritance	<code>extends</code>	Child gets the parent's traits
Encapsulation	<code>private</code> , getters/setters	Locked room with a doorman

Common gotchas

- **Overloading is not overriding.** Overloading is compile-time (same name, different parameters). Overriding is runtime (a subtype redefines a parent method, resolved by dynamic dispatch).
- **`private` fields are not visible to subclasses.** Use `protected` if a subclass needs direct access.
- **A class extends one class but implements many interfaces.**
- **Constructors are not inherited.** The child calls the parent through `super(...)` .
- **`final` blocks extension.** On a class it blocks inheritance, on a method it blocks overriding.
- **Interfaces are not just default methods.** Since Java 8 they can have `default` methods, since Java 9 also `static` and `private` methods. They still hold no instance state.
- **Composition often beats inheritance.** If you are extending only to reuse one method, hold the other class instead.

Interview traps

- Overloading is not overriding. Compile-time versus runtime.
- Encapsulation is not abstraction. Hides data versus hides implementation.
- Interface is not an abstract class. Contract versus shared partial code.
- Inheritance is not always better than composition. Prefer HAS-A when you can.
- Constructors are not inherited.

Abstract class versus interface, the rapid-fire round:

Question	Answer
Can an abstract class have a constructor?	Yes
Can an interface have a constructor?	No
Can an abstract class have implemented methods?	Yes
Can an interface have fields?	Only public static final constants

OOP in 30 seconds

Abstraction -> hide implementation
Polymorphism -> same call, different behaviour
Inheritance -> reuse through IS-A
Encapsulation -> hide data

Remember A PIE: Simplify -> Adapt -> Reuse -> Protect